



भारतीय सूचना प्रौद्योगिकी संस्थान गुवाहाटी
Indian Institute of Information Technology Guwahati

CS236 - ARTIFICIAL INTELLIGENCE LAB
SOLUTION - 02

1. Develop a Python program to interact with a text file named "example.txt". The program should perform the following operations:
 - i. Create a file named "example.txt" and populate it with some sample text. Ensure the file is saved in the same directory as your Python script.
 - ii. Read the entire content of "example.txt" using Python and display it.

```
def create_and_read_file(filename="example.txt", content="This is some sample text.\n"):
    """
    Creates a text file with the given filename and content,
    then reads and prints the entire content of the file.

    Args:
        filename (str): The name of the file to create and read.
        content (str): The text content to write to the file.
    """
    try:
        # Create and write to the file
        with open(filename, "w") as f:
            f.write(content)
        print(f"File '{filename}' created successfully.")

        # Read and display the file content
        with open(filename, "r") as f:
            file_content = f.read()
            print(f"\nContent of '{filename}':\n{file_content}")

    except Exception as e:
        print(f"An error occurred: {e}")

# Example usage:
create_and_read_file()
```

2. Write a Python program that reads a user-specified number of characters from a text file. Create a text file, and then your program should:
 - i. Prompt the user to enter the number of characters to read.
 - ii. Read and print the specified number of characters from the file.

```
def read_n_characters(filename, n):
    """
    Reads and prints the specified number of characters from a text file.
```

```

Args:
    filename (str): The name of the text file to read.
    n (int): The number of characters to read.
"""
try:
    with open(filename, "r") as f:
        characters = f.read(n)
        print(f"The first {n} characters of the file are:\n'{characters}'")
except FileNotFoundError:
    print(f"Error: File '{filename}' not found.")
except Exception as e:
    print(f"An error occurred: {e}")

# Create a sample text file (you can replace this with your own file)
sample_content = "This is a sample text file for testing.\nIt contains multiple lines of text."
with open("sample.txt", "w") as f:
    f.write(sample_content)

# Get input from the user
try:
    num_chars = int(input("Enter the number of characters to read: "))
    read_n_characters("sample.txt", num_chars)
except ValueError:
    print("Invalid input. Please enter an integer.")

```

3. Extend the program from Question 2 to append an additional line of text to the existing file. After appending, the program should display the entire updated content of the file.

```

def read_n_characters_and_append(filename, n):
    """
    Reads a specified number of characters from a file, appends a new line,
    and then displays the entire updated content.

    Args:
        filename (str): The name of the text file.
        n (int): The number of characters to read initially.
    """
    try:
        # Read the specified number of characters
        with open(filename, "r") as f:
            characters = f.read(n)
            print(f"The first {n} characters of the file are:\n'{characters}'")

        # Append a new line to the file
        new_line = input("Enter the line to append to the file: ")
        with open(filename, "a") as f:
            f.write("\n" + new_line) # Add a newline character before the appended line
        print(f"Line appended successfully to '{filename}'.")

        # Read and display the entire updated content
        with open(filename, "r") as f:
            updated_content = f.read()
            print(f"\nUpdated content of '{filename}':\n{updated_content}")

    except FileNotFoundError:
        print(f"Error: File '{filename}' not found.")
    except Exception as e:
        print(f"An error occurred: {e}")

```

```

# Create a sample text file (you can replace this with your own file)
sample_content = "This is a sample text file for testing.\nIt contains multiple lines of text."
with open("sample.txt", "w") as f:
    f.write(sample_content)

# Get input from the user for the number of characters to read
try:
    num_chars = int(input("Enter the number of characters to read: "))
    read_n_characters_and_append("sample.txt", num_chars)
except ValueError:
    print("Invalid input. Please enter an integer.")

```

4. Create a Python function named `count_lines_not_starting_with_T` that counts the number of lines in a text file named "text.txt" that do not begin with the letter "T". For example, assume "text.txt" contains the following lines:

A girl is playing there badminton.
 The scenery is beautiful.
 The birds are flying in the sky.
 The sky is cloudy.
 Alphabets consists of vowels and consonants.

```

def count_lines_not_starting_with_T(filename="text.txt"):
    """
    Counts the number of lines in a text file that do not begin with the letter 'T'.

    Args:
        filename (str): The name of the text file.

    Returns:
        int: The number of lines not starting with 'T'.
    """
    count = 0
    try:
        with open(filename, "r") as f:
            for line in f:
                if not line.strip().startswith("T"): # strip() removes leading/trailing whitespace
                    count += 1
            return count
    except FileNotFoundError:
        print(f"Error: File '{filename}' not found.")
        return -1 # Return -1 to indicate an error

# Create a sample "text.txt" file (replace with your own if needed)
sample_content = """A girl is playing there badminton.
The scenery is beautiful.
The birds are flying in the sky.
The sky is cloudy.
Alphabets consists of vowels and consonants."""
with open("text.txt", "w") as f:
    f.write(sample_content)

# Call the function and print the result
result = count_lines_not_starting_with_T()
if result != -1:
    print(f"The number of lines not starting with 'T' is: {result}")

```

5. Develop a Python program that reads the content of one text file and writes it to another. The program should:

- i. Read the content from an existing text file.
- ii. Create a new text file.
- iii. Write the content read from the first file into the newly created file.

```
def copy_file_content(source_filename, destination_filename):  
    """  
    Reads the content from a source text file and writes it to a new destination file.  
  
    Args:  
        source_filename (str): The name of the source file to read from.  
        destination_filename (str): The name of the destination file to create and write to.  
    """  
    try:  
        with open(source_filename, "r") as source_file:  
            content = source_file.read()  
  
        with open(destination_filename, "w") as destination_file:  
            destination_file.write(content)  
  
        print(f"Content from '{source_filename}' copied to '{destination_filename}' successfully.")  
  
    except FileNotFoundError:  
        print(f"Error: Source file '{source_filename}' not found.")  
    except Exception as e:  
        print(f"An error occurred: {e}")  
  
# Example usage:  
source_file = "source.txt" # Replace with your source file  
destination_file = "destination.txt" # The new file that will be created  
  
# Create a sample source file (replace with your own file if needed)  
with open(source_file, "w") as f:  
    f.write("This is the content of the source file.\n")  
    f.write("It has multiple lines.\n")  
  
copy_file_content(source_file, destination_file)
```

6. You are given a NumPy array (matrix) of dimensions $N \times M$ with space-separated integer elements. Your task is to calculate and display the transpose and the flattened version of this matrix.

- i. Input the matrix from the user or define it in your code.
- ii. Calculate and print the transpose of the matrix.
- iii. Calculate and print the flattened version of the matrix.

```
import numpy as np  
  
def transpose_and_flatten(matrix):  
    """  
    Calculates and prints the transpose and flattened version of a NumPy matrix.  
  
    Args:
```

```

        matrix (numpy.ndarray): The input NumPy matrix.
    """

    # Calculate the transpose
    transpose_matrix = matrix.T # Or np.transpose(matrix)

    # Calculate the flattened version
    flattened_matrix = matrix.flatten() # Or np.ravel(matrix)

    # Print the results
    print("Original Matrix:")
    print(matrix)
    print("\nTranspose of the Matrix:")
    print(transpose_matrix)
    print("\nFlattened Matrix:")
    print(flattened_matrix)

# Get matrix input from the user
try:
    rows = int(input("Enter the number of rows (N): "))
    cols = int(input("Enter the number of columns (M): "))

    matrix = []
    print("Enter the matrix elements row-wise (space-separated):")
    for i in range(rows):
        row = list(map(int, input().split()))
        if len(row) != cols:
            raise ValueError("Incorrect number of elements in a row.")
        matrix.append(row)

    # Convert the list of lists to a NumPy array
    np_matrix = np.array(matrix)

    transpose_and_flatten(np_matrix)

except ValueError as e:
    print(f"Invalid input: {e}")
except Exception as e:
    print(f"An error occurred: {e}")

```

7. Given a NumPy array of integers: `[10,16,71,9]`, `[71,91,31,51]`, write a program to determine and print the following:

- i. The dimension of the array.
- ii. The shape of the array.
- iii. The size (total number of elements) of the array.

```

import numpy as np

def array_info(arr):
    """
    Determines and prints the dimension, shape, and size of a NumPy array.

    Args:
        arr (numpy.ndarray): The input NumPy array.
    """

```

```

dimension = arr.ndim
shape = arr.shape
size = arr.size

print("Dimension of the array:", dimension)
print("Shape of the array:", shape)
print("Size of the array:", size)

# Given NumPy array
my_array = np.array([[10, 16, 71, 9], [71, 91, 31, 51]])

# Call the function to print the array information
array_info(my_array)

```

8. Create two random NumPy arrays of any dimension and perform the following operations:

- i. Concatenate the two arrays into a single array.
- ii. Sort both arrays individually.
- iii. Add the two arrays element-wise.
- iv. Subtract the second array from the first array element-wise.
- v. Multiply the two arrays element-wise.
- vi. Divide the first array by the second array element-wise (handle potential division by zero).

```

import numpy as np

def perform_array_operations(arr1, arr2):
    """
    Performs various operations on two NumPy arrays: concatenation, sorting,
    element-wise addition, subtraction, multiplication, and division.

    Args:
        arr1 (numpy.ndarray): The first NumPy array.
        arr2 (numpy.ndarray): The second NumPy array.
    """

    try:
        # i. Concatenate
        if arr1.shape == arr2.shape:
            concatenated_array = np.concatenate((arr1, arr2))
            print("\nConcatenated Array (along the first axis with same shape):")
            print(concatenated_array)
        else:
            concatenated_array = np.concatenate((arr1.flatten(), arr2.flatten()))
            print("\nConcatenated Array (after flattening):")
            print(concatenated_array)

        # ii. Sort
        sorted_arr1 = np.sort(arr1, axis=None) # Sort the flattened array
        sorted_arr2 = np.sort(arr2, axis=None)
        print("\nSorted Array 1:\n", sorted_arr1)
        print("\nSorted Array 2:\n", sorted_arr2)

        # iii. Add (element-wise)
        if arr1.shape == arr2.shape:

```

```

        add_result = arr1 + arr2
        print("\nElement-wise Addition:\n", add_result)
    else:
        print("\nElement-wise addition not possible due to incompatible shapes.")

    # iv. Subtract (element-wise)
    if arr1.shape == arr2.shape:
        subtract_result = arr1 - arr2
        print("\nElement-wise Subtraction:\n", subtract_result)
    else:
        print("\nElement-wise subtraction not possible due to incompatible shapes.")
    # v. Multiply (element-wise)
    if arr1.shape == arr2.shape:
        multiply_result = arr1 * arr2
        print("\nElement-wise Multiplication:\n", multiply_result)
    else:
        print("\nElement-wise multiplication not possible due to incompatible shapes.")

    # vi. Divide (element-wise)
    if arr1.shape == arr2.shape:
        # Handle division by zero using np.where()
        divide_result = np.where(arr2 != 0, arr1 / arr2, np.inf)
        print("\nElement-wise Division:\n", divide_result)
    else:
        print("\nElement-wise division not possible due to incompatible shapes.")

except ValueError as e:
    print(f"Error during operation: {e}")
except Exception as e:
    print(f"An unexpected error occurred: {e}")

# Create two random arrays of different dimensions
array1 = np.random.randint(1, 20, size=(2, 3)) # 2x3 array
array2 = np.random.randint(1, 10, size=(2, 3)) # 2x3 array

print("Array 1:\n", array1)
print("\nArray 2:\n", array2)

# Perform the operations
perform_array_operations(array1, array2)

```

9. Generate a random NumPy matrix of size 8×7 , where columns represent features and rows represent patterns. Find and display the maximum and minimum values within each feature (column).

```

import numpy as np

def find_max_min_per_feature(matrix):
    """
    Finds and displays the maximum and minimum values within each feature (column)
    of a NumPy matrix.

    Args:
        matrix (numpy.ndarray): The input NumPy matrix.
    """

    num_features = matrix.shape[1] # Get the number of columns (features)

```

```

for feature_index in range(num_features):
    column = matrix[:, feature_index] # Extract the current column
    max_value = np.max(column)
    min_value = np.min(column)

    print(f"Feature {feature_index + 1}:")
    print(f"  Maximum: {max_value}")
    print(f"  Minimum: {min_value}")

# Generate a random 8x7 matrix
matrix = np.random.rand(8, 7) # 8 rows, 7 columns, values between 0 and 1

print("Generated Matrix:\n", matrix)

# Find and display the maximum and minimum values per feature
find_max_min_per_feature(matrix)

```

10. Write a NumPy program to accomplish the following:

- i. Create a 4×5 matrix with values ranging from 1 to 20.
- ii. Create three arrays: one with 10 zeros, one with 10 ones, and one with 10 fives.
- iii. Create an array containing all even integers from 10 to 50.
- iv. Generate a single random number between 0 and 1.
- v. Save the 4×5 matrix created in part (i) to a text file named "matrix.txt". Then, load the matrix back from this file.

```

import numpy as np

# 1. Create a 4x5 matrix with values from 1 to 20
matrix_4x5 = np.arange(1, 21).reshape(4, 5)
print("\n1. 4x5 Matrix:\n", matrix_4x5)

# 2. Create arrays of zeros, ones, and fives
zeros_array = np.zeros(10)
ones_array = np.ones(10)
fives_array = np.full(10, 5)
print("\n2. Arrays:")
print("  Zeros:", zeros_array)
print("  Ones:", ones_array)
print("  Fives:", fives_array)

# 3. Create an array of even integers from 10 to 50
even_numbers = np.arange(10, 51, 2)
print("\n3. Even numbers from 10 to 50:", even_numbers)

# 4. Generate a random number between 0 and 1
random_number = np.random.rand()
print("\n4. Random number between 0 and 1:", random_number)

# 5. Save and load the 4x5 matrix
# Save
np.savetxt("matrix.txt", matrix_4x5)
print("\n5. Matrix saved to 'matrix.txt'")

# Load
loaded_matrix = np.loadtxt("matrix.txt")
print("  Matrix loaded from 'matrix.txt':\n", loaded_matrix)

```